Experimental Analysis of Sorting Algorithms Based on Time Complexity

Dragoș Glăv
Department of Computer Science,
West University Timișoara
email: dragos.glav06@e-uvt.ro

May 5, 2026

Abstract

This paper presents an experimental study of classical sorting algorithms, including Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort, and Radix Sort. The aim is to analyze and compare their performance based on execution time under different input conditions.

The algorithms are implemented in C++ and tested on datasets of various sizes and distributions, such as random, sorted, reverse-sorted, and almost sorted arrays. The results confirm the theoretical time complexities and highlight differences in real-world performance.

The study provides a practical comparison that helps to understand which sorting algorithms are more efficient depending on the input characteristics.

Contents

1 Introduction

In Computer Science, sorting is one of the most fundamental problems. Given a list of elements, the objective is to arrange them in the preferred order (ascending / descending). However, efficiency is what significantly escalates this problem. Sorting is widely used in databases, search engines, data analysis, scheduling systems, and many other software applications.

Several sorting algorithms have been created, such as Bubble Sort, Insertion Sort, and Selection Sort. These methods differ both by difficulty to implement and time complexity.

Sorting algorithms are an important asset in Computer Science, as many tasks (for example: searching, duplicate detection) become much more efficient if the data is sorted.

However, existing sorting algorithms are not classified in a hierarchical way, so we cannot name one specific algorithm as the most efficient one. For example, a very efficient algorithm such as Quick Sort, will have unfavorable results in certain cases, while an algorithm like Merge Sort requires extra memory. Therefore, choosing the most appropriate algorithm remains an important problem.

In order to address this problem, this paper focuses on the analysis of different sorting algorithms based on Time Complexity, while analyzing special cases. The study focuses on several classical sorting algorithms, including Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort, and Radix Sort. The proposed approach consists of an experimental evaluation, in which the assessed algorithms are implemented and tested on several different cases. The purpose is to compare them and trace conclusions. More advanced algorithms often achieve better asymptotic complexity, but may involve trade-offs such as higher implementation complexity or memory usage. However, as mentioned before, even advanced algorithms can have disappointing results in certain cases. This paper investigates these aspects.

As to give an illustration of the problem, let us give an example. Consider the array $A = [7, 4, 6, 2, 9]$, which can be sorted into $[2, 4, 6, 7, 9]$. Different sorting algorithms achieve this task using distinct strategies and exhibit different time complexities. For instance, Bubble Sort will compare items next to one another, but the process is very inefficient. On the other hand, Quick Sort will achieve the result very efficiently, but can degrade significantly depending on the list.

The contribution of this paper is the implementation and comparison of several sorting algorithms. The study includes both theoretical analysis and practical testing on different datasets. The paper also highlights the differences between algorithms in terms of speed and efficiency, helping to understand which methods are more suitable in different situations.

The remainder of this paper is structured as follows. Section 2 provides a formal description of the sorting problem and introduces the algorithms considered in this study, along with their theoretical properties. Section 3 presents the implementation details and the system model used for experimentation. Section 4 is dedicated to experimental evaluation, where the performance of the algorithms is analyzed and compared based on execution time and scalability.

Section 5 discusses related work and places the results in the context of existing research. Finally, Section 6 concludes the paper and outlines possible directions for future work.

2 Sorting Problem and Algorithm Analysis

2.1 Problem Definition

Let $A = [a_1, a_2, \ldots, a_n]$ be a sequence of n elements defined over a totally ordered set. The sorting problem consists of finding a permutation π such that:

$$a_{\pi(1)} \leq a_{\pi(2)} \leq \ldots \leq a_{\pi(n)}$$

2.2 Sorting Algorithms

Several classical sorting algorithms are considered in this study, each based on different computational strategies and exhibiting different performance characteristics.

Bubble Sort

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.

Time Complexity:

- Best case: $O(n)$
- Average case: $O(n^2)$
- Worst case: $O(n^2)$

Insertion Sort

Insertion Sort builds the sorted array incrementally by inserting each element into its correct position.

Time Complexity:

- Best case: $O(n)$
- Average case: $O(n^2)$
- Worst case: $O(n^2)$

Selection Sort

Selection Sort repeatedly selects the minimum element from the unsorted portion and places it at the beginning.

Time Complexity:

- Best case: $O(n^2)$
- Average case: $O(n^2)$
- Worst case: $O(n^2)$

Merge Sort

Merge Sort is a divide-and-conquer algorithm that recursively splits the array and merges sorted subarrays.

Time Complexity:

- Best case: $O(n \log n)$
- Average case: $O(n \log n)$
- Worst case: $O(n \log n)$

Quick Sort

Quick Sort partitions the array around a pivot and recursively sorts the subarrays.

Time Complexity:

- Best case: $O(n \log n)$
- Average case: $O(n \log n)$
- Worst case: $O(n^2)$

Radix Sort

Radix Sort is a non-comparison-based algorithm that sorts elements digit by digit, starting from the least significant digit.

Time Complexity:

- Best case: $O(nk)$
- Average case: $O(nk)$
- Worst case: $O(nk)$

where k is the number of digits of the largest number.

Example

Consider the array:

$$A = [5, 2, 9, 1, 6]$$

The sorted result is:

$$A_{sorted} = [1, 2, 5, 6, 9]$$

Each algorithm achieves this result using different computational strategies and efficiency levels.

3 Implementation of Sorting Algorithms

The sorting problem is modeled in a computational context as the processing of an array of numerical values stored in memory. The goal is to transform an unsorted array into a sorted one using different algorithmic strategies. In this implementation, the problem is represented using standard data structures such as arrays (or vectors), which allow efficient access and modification of elements.

The solution has been implemented in a high-level programming language (C++) using a modular approach. Each sorting algorithm is implemented as a separate function, which allows independent testing and comparison. The implemented algorithms include Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort, and Radix Sort. This modular design improves readability, maintainability, and extensibility of the code.

For comparison purposes, a common interface is used where each algorithm receives the same input array and returns a sorted version of it. Execution time is measured using standard timing utilities provided by the programming environment, allowing performance evaluation under identical conditions.

From an architectural perspective, the system consists of three main components: data generation, algorithm execution, and performance measurement. The data generation module creates test arrays of different sizes and characteristics, such as random, sorted, and reverse-sorted inputs. The execution module applies each sorting algorithm to the generated data, while the measurement module records the execution time for analysis.

From a user perspective, the program is simple to use. The user specifies the size of the input array and the type of dataset to be generated. The program then automatically applies all implemented sorting algorithms and outputs the sorted results along with the execution time for each algorithm. The output is displayed in the console in a structured format, allowing easy comparison between methods.

Overall, this implementation provides a practical framework for analyzing and comparing sorting algorithms in a controlled computational environment.

4 Case Studies and Experimental Evaluation

In this section, the practical behavior of the implemented sorting algorithms is analyzed through a series of experiments. The goal is to evaluate and compare their performance under different input conditions and to validate the theoretical complexity results presented in the previous section.

The experiments were conducted using a C++ implementation developed in Visual Studio Code. Execution time was measured using high-resolution timing utilities to ensure accuracy and consistency. Each algorithm was executed multiple times on the same input sizes in order to reduce measurement noise, and the average execution time was recorded.

Three types of input datasets were used in the evaluation process: randomly generated arrays, already sorted arrays, and reverse-sorted arrays. These cases were selected in order to analyze best-case, average-case, and worst-case behavior of the algorithms.

The size of the input arrays varied from small-scale inputs (100 elements) to large-scale inputs (up to 50,000 elements). For each input size, all implemented algorithms (Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort, and Radix Sort) were executed under identical conditions.

Algorithm	50	100	1000	5000	10000
Bubble Sort	3	18	1990	-	-
Selection Sort	2	10	987	-	-
Insertion Sort	1	5	529	-	-
Quick Sort	1	2	71	498	1069
Merge Sort	15	32	488	2596	5090
Radix Sort	7	14	150	634	1247

Table 1: Execution time (μs) for random arrays

The results for random input arrays provide a basis for comparing the performance of all implemented sorting algorithms. Quadratic algorithms such as Bubble Sort, Selection Sort, and Insertion Sort exhibit significantly higher execution times as the input size increases, confirming their $O(n^2)$ complexity.

In contrast, Quick Sort, Merge Sort, and Radix Sort demonstrate substantially better scalability and lower execution times for larger datasets.

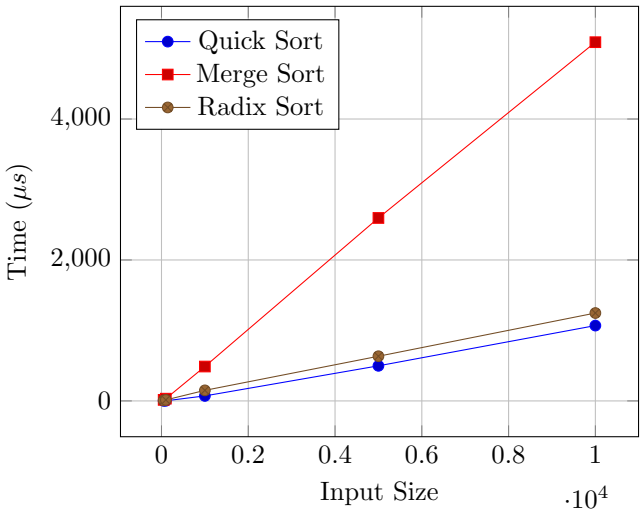

Figure 1: Performance comparison on random input arrays

The graphical representation confirms the trends observed in the tabular data. It clearly shows the superior scalability of Quick Sort among comparison-based algorithms, as well as the stable growth of Merge Sort and Radix Sort.

Algorithm	50	100	1000	5000	10000
Bubble Sort	2	8	1023	-	-
Selection Sort	2	10	940	-	-
Insertion Sort	0	0	2	-	-
Quick Sort	6	26	2479	60955	236440
Merge Sort	13	28	387	2092	4563
Radix Sort	2	5	72	504	1118

Table 2: Execution time (μs) for sorted arrays

The results for sorted input arrays reveal important differences in algorithm behavior under best-case conditions. Insertion Sort performs extremely efficiently, achieving nearly constant execution time due to its linear behavior on already sorted data.

Bubble Sort and Selection Sort still exhibit relatively high execution times compared to adaptive algorithms, showing limited improvement even in favorable conditions.

Interestingly, Quick Sort performs worse on sorted data compared to random inputs, indicating sensitivity to pivot selection and partition imbalance. In contrast, Merge Sort and Radix Sort maintain stable performance regardless of input ordering.

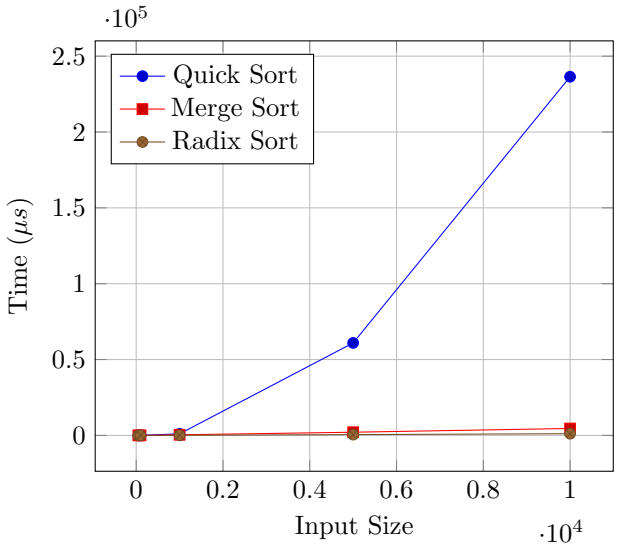

Figure 2: Performance comparison on sorted input arrays

The graph further emphasizes the differences observed in the experimental data. It clearly shows the significant degradation of Quick Sort performance on sorted inputs, while Merge Sort and Radix Sort maintain consistent growth trends.

Algorithm	50	100	1000	5000	10000
Bubble Sort	6	26	2376	-	-
Selection Sort	2	12	1063	-	-
Insertion Sort	2	12	1079	-	-
Quick Sort	5	18	1679	42841	160077
Merge Sort	13	31	370	2087	4118
Radix Sort	2	10	98	469	1312

Table 3: Execution time (μs) for reverse-sorted arrays

The results for reverse-sorted arrays represent a worst-case scenario for several sorting algorithms. Bubble Sort, Selection Sort, and Insertion Sort show significantly increased execution times, confirming their sensitivity to input order.

Quick Sort exhibits a major performance degradation, indicating poor pivot behavior in adversarial input conditions. In contrast, Merge Sort and Radix Sort maintain relatively stable execution times, demonstrating robustness regardless of input arrangement.

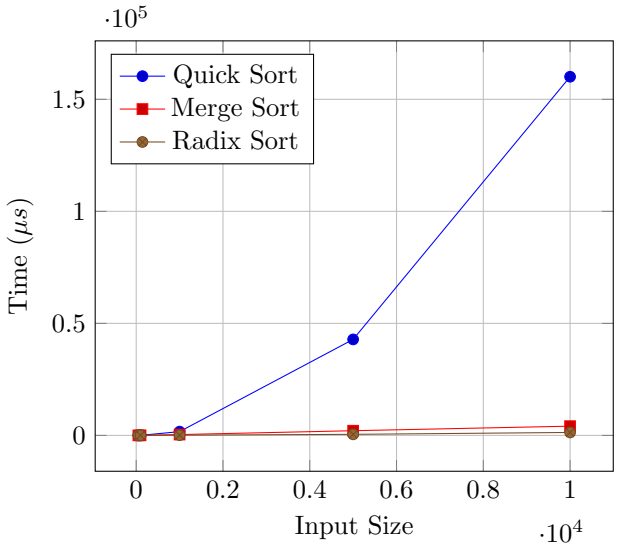

Figure 3: Performance comparison on reverse-sorted arrays

The graph highlights the significant impact of reverse ordering on Quick Sort performance, showing exponential growth in execution time for large inputs.

Merge Sort and Radix Sort maintain stable behavior, confirming their independence from input ordering and consistent theoretical complexity.

Algorithm	50	100	1000	5000	10000
Bubble Sort	2	9	1038	-	-
Selection Sort	2	9	921	-	-
Insertion Sort	0	0	32	-	-
Quick Sort	7	19	426	1506	2979
Merge Sort	15	34	383	2175	4286
Radix Sort	3	5	72	543	1073

Table 4: Execution time (μs) for almost sorted arrays

The results for almost sorted arrays demonstrate the adaptive behavior of certain algorithms. Insertion Sort performs efficiently in this scenario due to the small number of displaced elements.

Bubble Sort shows moderate improvement compared to worst-case inputs, while Selection Sort remains relatively inefficient.

Quick Sort performs significantly better than in reverse-sorted cases, suggesting improved partition balance. Merge Sort and Radix Sort maintain stable execution times across all input sizes.

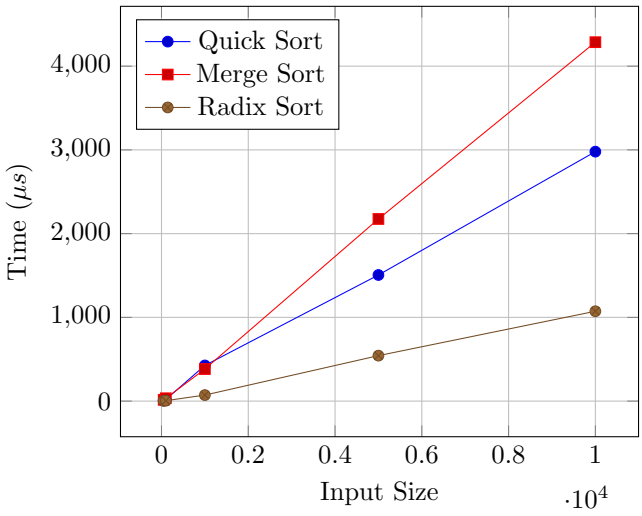

Figure 4: Performance comparison on almost sorted arrays

The graphical representation confirms that almost sorted inputs significantly benefit adaptive algorithms such as Quick Sort and Insertion Sort.

The differences between algorithms become less pronounced compared to worst-case scenarios, although Merge Sort and Radix Sort continue to exhibit stable growth patterns.

Algorithm	50	100	1000	5000	10000
Bubble Sort	3	14	1686	-	-
Selection Sort	2	12	986	-	-
Insertion Sort	1	3	474	-	-
Quick Sort	2	6	530	12201	46750
Merge Sort	14	31	495	2306	4680
Radix Sort	1	2	33	129	260

Table 5: Execution time (μs) for arrays with few distinct values

The results for arrays with few distinct values highlight the impact of data distribution on sorting performance.

Quick Sort shows reduced efficiency due to increased likelihood of unbalanced partitions when many duplicate values are present.

Radix Sort performs particularly well in this case, benefiting from its digit-based grouping strategy. Merge Sort remains stable, while quadratic algorithms continue to perform poorly for large input sizes.

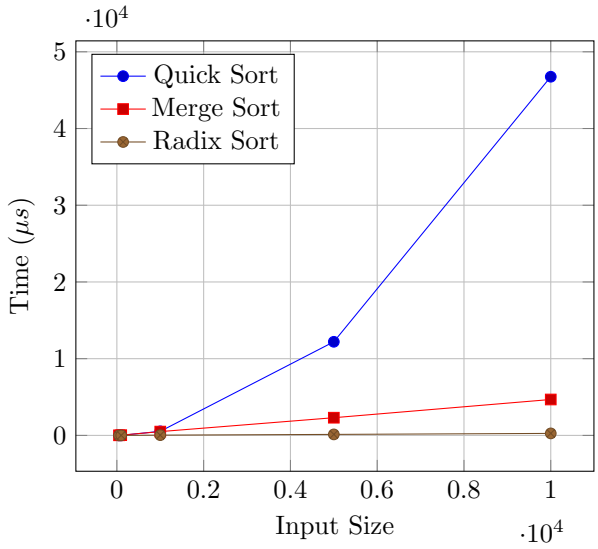

Figure 5: Performance comparison on arrays with few distinct values

The graph emphasizes the advantage of Radix Sort in handling datasets with limited value diversity.

Quick Sort performance decreases due to less effective partitioning, while Merge Sort maintains consistent scalability. The visualization reinforces the influence of data distribution on algorithm efficiency.

Overall, the experimental results confirm that the performance of sorting algorithms strongly depends on both input size and data characteristics. While Quick Sort achieves very good performance on average, its execution time increases dramatically for sorted and reverse-sorted inputs, confirming its sensitivity to pivot selection.

Merge Sort demonstrates stable and predictable behavior across all test cases, making it a reliable choice regardless of input distribution. Radix Sort also shows consistently good performance, particularly for large datasets with integer values, due to its non-comparison-based approach.

In contrast, quadratic algorithms such as Bubble Sort, Insertion Sort, and Selection Sort become impractical for large input sizes, which explains their omission from tests involving 5000 and 10000 elements.

The experimental setup is reproducible, as all algorithms are executed under identical conditions using the same input data. This ensures that the comparisons are fair and that the results accurately reflect the practical behavior of each algorithm.

5 Related Work

Sorting algorithms have been extensively studied in the field of computer science [1, 2], both from theoretical and experimental perspectives. Classical textbooks such as *Introduction to Algorithms* [1] provide a comprehensive analysis of fundamental sorting algorithms, including Merge Sort, Quick Sort, and Heap Sort, along with their asymptotic time and space complexities.

Numerous studies have focused on the empirical evaluation of sorting algorithms in order to better understand their practical performance beyond theoretical bounds. For instance, experimental comparisons have shown that although Quick Sort has an average time complexity of $O(n \log n)$, its performance is highly dependent on pivot selection and input distribution, which may lead to significant degradation in unfavorable cases. Various optimizations, such as randomized pivot selection and median-of-three strategies, have been proposed to address these limitations.

In contrast, Merge Sort has been widely recognized for its stable performance and guaranteed $O(n \log n)$ complexity regardless of input characteristics, at the cost of additional memory usage. Similarly, non-comparison-based algorithms such as Radix Sort have been investigated for their ability to achieve linear time complexity under specific conditions, particularly when sorting integers or fixed-length keys.

Recent research has also explored advanced sorting techniques, including parallel and distributed algorithms designed to leverage multi-core processors and GPU architectures. These approaches aim to significantly reduce execution time for large-scale datasets, although they often introduce additional implementation complexity and hardware dependencies.

Compared to existing work, the contribution of this paper lies in a systematic experimental evaluation of several classical sorting algorithms under different input conditions, including random, sorted, reverse-sorted, and partially sorted datasets. While many studies focus primarily on theoretical analysis or specialized implementations, this work emphasizes practical performance in a controlled and reproducible environment using standard programming tools.

Furthermore, this study highlights the impact of input distribution on algorithm efficiency and provides a direct comparison between comparison-based and non-comparison-based methods. The results obtained are consistent with established theoretical expectations, while also offering additional insights into real-world behavior and performance trade-offs.

6 Conclusions and Future Work

This paper addressed the problem of sorting data efficiently by analyzing and comparing several classical sorting algorithms. The study included both theoretical analysis and practical implementation of Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort, and Radix Sort.

The experimental results confirmed the theoretical expectations regarding

algorithm performance. Quadratic algorithms such as Bubble Sort, Insertion Sort, and Selection Sort proved to be inefficient for large input sizes, while more advanced algorithms such as Merge Sort and Quick Sort demonstrated significantly better scalability. Radix Sort also showed strong performance, particularly for integer data, highlighting the advantages of non-comparison-based methods.

During the implementation and testing process, several challenges were encountered. One of the main difficulties was ensuring fair and accurate measurement of execution time, especially for very fast algorithms where differences are small. This issue was addressed by running multiple tests and considering average values. Another challenge was handling large input sizes efficiently without introducing external factors that could affect performance.

Despite the successful implementation and evaluation, some limitations remain. The experiments were conducted in a single programming environment and did not consider factors such as memory usage in detail or performance on different hardware architectures. Additionally, only sequential versions of the algorithms were analyzed.

Future work may include extending the study to parallel and distributed sorting algorithms, as well as testing performance on different platforms and datasets. Further analysis could also consider memory consumption and stability in more depth. Another direction is the optimization of existing implementations and the exploration of hybrid sorting techniques that combine multiple strategies.

Overall, the paper provides a practical and comparative perspective on sorting algorithms and highlights their behavior in real-world scenarios.

Bibliography

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed., Cambridge, MA, USA: MIT Press, 2009.

[2] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed., Reading, MA, USA: Addison-Wesley, 1998.

[3] C. A. R. Hoare, "Quicksort," *The Computer Journal*, vol. 5, no. 1, pp. 10–16, 1962.

[4] M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed., Pearson, 2014.

[5] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed., Addison-Wesley, 2011.